Manuel de Référence Utilisateur et Guide Pratique - Solveur COVOF 2D

Guide Complet d'Installation, Paramétrage, Exécution et Post-Traitement

Version 2.1.0 — Conforme aux Standards du Laboratoire TREFLE / I2M (Référence Académique)

Sommaire Général

1. Présentation et Fonctionnalités du Logiciel
   — 1.1. Philosophie du Solveur COVOF 2D
   — 1.2. Matrice Globale des Fonctionnalités Physiques et Numériques
2. Installation, Dépendances et Compilation
   — 2.1. Prérequis Système et Compilateurs C++20
   — 2.2. Bibliothèques Externes (MPI, Python, Outils de Tracé)
   — 2.3. Procédure de Compilation pas-à-pas avec CMake
   — 2.4. Arborescence et Organisation des Fichiers du Projet
3. Format Complet des Fichiers de Configuration (JSON)
   — 3.1. Structure Générale d'un Cas Test
   — 3.2. Dictionnaire Exhaustif des Modules et Mots-Clés
      — 3.2.1. Module `mesh` (Maillage et Dimensions Géométriques)
      — 3.2.2. Module `physics` (Propriétés Physiques et Modèles d'Écoulement)
      — 3.2.3. Module `thermal` (Couplage Énergétique et Boussinesq)
      — 3.2.4. Module `time` (Intégration Temporelle et Contrôle CFL)
      — 3.2.5. Module `boundary_conditions` (Conditions aux Limites Fluides et Thermiques)
      — 3.2.6. Module `immersed_boundary` (Frontières Immergées et Pénalisation Brinkman)
      — 3.2.7. Module `initial_condition` (Champs Initiaux et Géométries d'Interface)
      — 3.2.8. Module `solver` (Méthode Numérique, Solveur Krylov et Préconditionneur)
      — 3.2.9. Module `output` (Sorties Graphiques VTK et Diagnostics CSV)
4. Tutoriel pas-à-pas : Construire un Cas de Simulation
   — 4.1. Tutoriel 1 : Écoulement Monophasique dans une Cavité Entraînée ($Re = 1000$)
   — 4.2. Tutoriel 2 : Convection Naturelle Thermique de Rayleigh-Bénard / Vahl Davis
   — 4.3. Tutoriel 3 : Écoulement Diphasique à Surface Libre (Rupture de Barrage)
5. Exécution du Code en Séquentiel et Parallèle (OpenMPI)
   — 5.1. Lancement Local sur Poste de Travail ou Ordinateur Portable
   — 5.2. Lancement sur Supercalculateur et Gestionnaire de Tâches SLURM
   — 5.3. Décomposition de Domaine MPI et Équilibrage de Charge
6. Suivi d'Exécution et Diagnostic des Calculs en Temps Réel
   — 6.1. Interprétation du Tableau de Bord Console
   — 6.2. Détection Précoce des Instabilités et Divergences
   — 6.3. Guide de Dépannage et Stratégies de Remédiation
7. Post-Traitement et Analyse Graphique avec ParaView
   — 7.1. Chargement des Fichiers `.pvtr` et `.vtr`
   — 7.2. Extraction des Lignes de Courant et Profils de Vitesse
   — 7.3. Tracé des Isothermes et Isosurfaces d'Interface ($C = 0.5$)
   — 7.4. Calcul de la Vorticité et des Gradients de Pression
8. Automatisation des Benchmarks et Scripts de Vérification
   — 8.1. Le Script d'Orchestration Globale `run_all_benchmarks.sh`
   — 8.2. Vérification Statistique Automatisée `verify_cases.py`
   — 8.3. Production Automatique des Figures de Rapport `generate_validation_figures.py`
9. Foire Aux Questions (FAQ) et Astuces Pratiques

---

1. Présentation et Fonctionnalités du Logiciel

1.1. Philosophie du Solveur COVOF 2D

Le code **COVOF 2D** est un environnement complet de simulation numérique des écoulements de fluides incompressibles en deux dimensions d'espace. Il a été conçu avec les impératifs d'ingénierie logicielle suivants : - **Fidélité physique maximale** : Implémentation stricte des équations de Navier-Stokes à viscosité et masse volumique variables, de l'approximation de Boussinesq pour la thermique, du modèle CSF de Brackbill pour les interfaces et de la pénalisation volumique de Brinkman pour les obstacles. - **Robustesse algorithmique absolue** : Utilisation de la méthode du **Lagrangien Augmenté**, garantissant l'incompressibilité sans instabilités de pression et éliminant le besoin de résoudre une équation de Poisson mal conditionnée. - **Hautes performances et montée en charge** : Parallélisme natif via **OpenMPI**, décomposition de domaine cartésienne 2D équilibrée, communications non bloquantes, structures de données à accès mémoire contigu sans aucune allocation dynamique pendant la boucle en temps. - **Simplicité d'utilisation** : Fichiers d'entrée au format standard **JSON**, entièrement autodocumentés, sans compilation préalable pour changer de géométrie, de maillage ou de conditions physiques. - **Interfaçage standardisé** : Sorties graphiques directes au format XML VTK multi-blocs (`.pvtr` et `.vtr`), visualisables immédiatement sous ParaView ou VisIt sans utilitaire de conversion.

1.2. Matrice Globale des Fonctionnalités Physiques et Numériques

Domaine	Fonctionnalité Disponible	Séquentiel	Parallèle (OpenMPI)	Détails Techniques & Référence
Écoulement	Monophasique incompressible			Navier-Stokes avec tenseur visqueux complet
	Viscosité variable continue			Modèle exponentiel ou analytique nodale
	Thermoconvection naturelle			Boussinesq, nombre de Nusselt automatisé
	Diphasique non miscible			VOF (Volume of Fluid) conservatif
	Tension superficielle			Modèle CSF Brackbill Balanced-Force
	Frontières immergées			Pénalisation Brinkman (marche, cylindre, L)
Maillage	Cartésien MAC 2D décalé			Staggered grid de Harlow & Welch (1965)
	Périodicité			Raccord torique d'indices et échanges de halos
Solveurs	BiCGSTAB(2)			Solveur Krylov résiduel d'ordre 2 (Sleijpen)
	BiCGSTAB standard			Gradient biconjugué stabilisé
	Gradient Conjugué (PCG)			Dédié aux matrices symétriques définies positives

Domaine	Fonctionnalité Disponible	Séquentiel	Parallèle (OpenMPI)	Détails Techniques & Référence
	GMRES(m)			Redémarrages avec rotations de Givens
	MINRES			Pour systèmes symétriques indéfinis
	MUMPS / PETSc / HYPRE	Option	Option	Interfaces prêtes pour solveurs externes directs/AMG
Préconditionnement	Jacobi (Diagonal)			Parallélisme parfait sans communication inter-cœurs
	SSOR / ILU(0)			Factorisation incomplète par sous-domaine

---

2. Installation, Dépendances et Compilation

2.1. Prérequis Système et Compilateurs C++20

COVOF 2D exploite les fonctionnalités avancées du standard **C++20** (concepts, vues, attributs d'optimisation). - **Systèmes d'exploitation supportés** : Linux (Ubuntu 20.04+, RHEL 8+, Debian 11+), macOS (macOS 12+ avec Clang/AppleClang ou GCC Homebrew). - **Compilateurs C++ certifiés** : - `GCC` version 10.3 ou ultérieure (version 11+ recommandée). - `Clang / LLVM` version 13.0 ou ultérieure. - `AppleClang` version 13.0 ou ultérieure sous macOS.

2.2. Bibliothèques Externes

1. **CMake** : Version ≥ 3.20.
2. **OpenMPI** : Version ≥ 4.0 (ou MPICH ≥ 3.4). Fournit les exécutables `mpicxx` et `mpirun`.
3. **Python 3** : Version ≥ 3.8 avec les paquets scientifiques standards :

```
pip install numpy matplotlib scipy
```

2.3. Procédure de Compilation pas-à-pas avec CMake

Pour compiler COVOF 2D en mode optimisé pour la production :

```
# 1. Se positionner à la racine du projet
cd COVOF_2D-FORTRAN

# 2. Créer le répertoire de compilation isolé
mkdir -p build && cd build

# 3. Configurer le projet avec CMake en mode Release (optimisations -O3)
cmake .. -DCMAKE_BUILD_TYPE=Release -DCMAKE_CXX_COMPILER=mpicxx

# 4. Compiler sur plusieurs cœurs (ex: 4 cœurs)
make -j4

# 5. Vérifier la génération de l'exécutable
```

```
ls -lh ../bin/covof2d
cd ..
```

L'exécutable final est automatiquement placé dans le répertoire `bin/covof2d`.

2.4. Arborescence et Organisation des Fichiers du Projet

```
covof2d/
   CMakeLists.txt              # Configuration de compilation CMake
   bin/                        # Exécutable binaire final (covof2d)
   include/covof/               # En-têtes C++ de la bibliothèque COVOF
       Config.hpp             # Structure de données de configuration
       Fields2D.hpp           # Allocation mémoire des variables scalaires et vectorielles
       InterfaceTracker.hpp   # Algorithmes VOF et modèle CSF capillaire
       LinearSolver.hpp       # Hiérarchie des solveurs de Krylov et préconditionneurs
       Mesh2D.hpp             # Grille cartésienne MAC décalée et métriques
       MpiComm.hpp            # Topologie cartésienne MPI et échanges de halos
       NavierStokesSolver.hpp # Assemblage discret, Lagrangien Augmenté, Thermique
       Simulation.hpp         # Orchestration de l'avancement temporel
       VTKWriter.hpp          # Génération des fichiers XML VTK pour ParaView
   src/                        # Fichiers sources C++ implémentant les modules
       Fields2D.cpp
       InterfaceTracker.cpp
       LinearSolver.cpp
       Mesh2D.cpp
       MpiComm.cpp
       NavierStokesSolver.cpp
       Simulation.cpp
       VTKWriter.cpp
       main.cpp               # Point d'entrée principal de l'exécutable
   cases/                      # Fichiers de configuration JSON des benchmarks
   docs/                       # Manuels complets de théorie, utilisateur, dev, validation
       MATHEMATICAL_THEORY.md # Notice théorique et mathématique
       USER_GUIDE.md          # Ce manuel utilisateur
       DEVELOPER_GUIDE.md     # Guide d'architecture logicielle C++
       VALIDATION_GUIDE.md    # Cahier de validation et vérification Référence Académique CFD
       figures/               # Figures haute définition des validations
   scripts/                    # Scripts d'automatisation et de post-traitement
       run_all_benchmarks.sh  # Lance la suite complète des 13 cas tests
       verify_cases.py        # Analyse statistique et vérification des critères d'erreur
       generate_validation_figures.py # Générateur de figures PNG
```

---

3. Format Complet des Fichiers de Configuration (JSON)

3.1. Structure Générale d'un Cas Test

Chaque simulation est entièrement pilotée par un fichier texte structuré selon le standard **JSON**. Un fichier type contient neuf blocs fonctionnels :

```
{
  "case_name": "cavity_re1000",
  "mesh": { ... },
  "physics": { ... },
  "thermal": { ... },
  "time": { ... },
  "boundary_conditions": { ... },
  "immersed_boundary": { ... },
```

```
  "initial_condition": { ... },
  "solver": { ... },
  "output": { ... }
}
```

3.2. Dictionnaire Exhaustif des Modules et Mots-Clés

3.2.1. Module `mesh` (Maillage Cartésien) Définit la géométrie du domaine rectangulaire et le nombre de cellules de calcul : - `nx` (entier, > 0) : Nombre de cellules de contrôle dans la direction axiale X. - `ny` (entier, > 0) : Nombre de cellules de contrôle dans la direction transversale Y. - `lx` (réel, > 0) : Longueur physique du domaine selon X [m]. - `ly` (réel, > 0) : Hauteur physique du domaine selon Y [m]. - `origin_x` (réel, optionnel, défaut : `0.0`) : Abscisse du coin inférieur gauche [m]. - `origin_y` (réel, optionnel, défaut : `0.0`) : Ordonnée du coin inférieur gauche [m].

3.2.2. Module `physics` (Propriétés et Modèles d'Écoulement) Contrôle les modèles physiques activés et les propriétés de fluide : - `enable_convection` (booléen) : Active (`true`) ou désactive (`false`) les termes de transport convectif $(\mathbf{u} \cdot \nabla)\mathbf{u}$. Doit être `false` pour les écoulements de Stokes purs ou le problème de Szymanski. - `convective_scheme` (chaîne, défaut : `"UPWIND"`) : Schéma numérique pour la convection : - `"UPWIND"` : Schéma décentré amont d'ordre 1, inconditionnellement stable. - `"CENTERED"` : Schéma centré d'ordre 2, non dissipatif. - `"TVD_SUPERBEE"` : Schéma d'ordre 2 avec limiteur de flux Superbee. - `gx` (réel, défaut : `0.0`) : Accélération de la pesanteur selon X [m/s^2]. - `gy` (réel, défaut : `0.0`) : Accélération de la pesanteur selon Y [m/s^2] (ex : `-9.81` pour la gravité terrestre). - `fluid1` (objet) : Propriétés thermodynamiques de la phase principale : - `rho` (réel, > 0) : Masse volumique [kg/m^3]. - `mu` (réel, > 0) : Viscosité dynamique [Pa $\cdot$ s]. - `enable_two_phase` (booléen, défaut : `false`) : Active le modèle diphasique à deux fluides immiscibles. - `fluid2` (objet, requis si `enable_two_phase = true`) : Propriétés de la seconde phase (`rho`, `mu`). - `enable_surface_tension` (booléen, défaut : `false`) : Active le modèle de tension de surface CSF de Brackbill. - `sigma` (réel, ≥ 0) : Coefficient de tension de surface [N/m]. - `viscosity_model` (chaîne, défaut : `"CONSTANT"`) : - `"CONSTANT"` : Viscosité spatiale homogène par phase. - `"EXPONENTIAL"` : Viscosité spatialement variable continue selon la loi :

$$\mu(y) = \mu_0 \exp\left(a \frac{(y-e)^2}{e^2}\right)$$

- `viscosity_exp_mu0`, `viscosity_exp_a`, `viscosity_exp_e` (réels) : Paramètres du modèle exponentiel.

3.2.3. Module `thermal` (Couplage Énergétique et Boussinesq) Contrôle la résolution de l'équation de la chaleur et la force de flottabilité : - `enable_energy` (booléen, défaut : `false`) : Active la résolution de l'équation d'énergie thermique pour le champ de température T. - `beta` (réel, défaut : `0.0`) : Coefficient de dilatation volumique thermique [K^{-1}]. - `t_ref` (réel, défaut : `0.0`) : Température de référence de Boussinesq [K] ou [°C]. - `t_hot` (réel) : Température imposée à la paroi chaude [K]. - `t_cold` (réel) : Température imposée à la paroi froide [K]. - `k_thermal` (réel, > 0, défaut : `1.0`) : Conductivité thermique [W/(m $\cdot$ K)]. - `cp_thermal` (réel, > 0, défaut : `1.0`) : Capacité thermique isobare [J/(kg $\cdot$ K)].

3.2.4. Module `time` (Intégration Temporelle)

- `dt` (réel, > 0) : Pas de temps d'intégration [s].
- `t_final` (réel, > 0) : Temps physique maximal de la simulation [s].
- `adaptive_dt` (booléen, défaut : `false`) : Active la régulation automatique du pas de temps selon la condition CFL.
- `cfl_max` (réel, défaut : `0.5`) : Nombre CFL limite admissible.

3.2.5. Module `boundary_conditions` (Conditions aux Limites) Définit les conditions physiques sur les quatre frontières cardinales : `left`, `right`, `bottom`, `top`. Chaque frontière comporte : - `type` (chaîne) : - `"DIRICHLET"` : Vitesse imposée à la frontière. Requièrt les champs `u` et `v`. - `"FREE_SLIP"` : Vitesse normale nulle, contrainte de cisaillement nulle (symétrie). - `"NEUMANN"` : Flux de vitesse normal nul ($\partial \mathbf{u}/\partial n = \mathbf{0}$, sortie libre). - `"PERIODIC"` : Périodicité spatiale liant les faces opposées. - `thermal_type` (chaîne, optionnel si la thermique est activée) : - `"DIRICHLET"` : Température imposée égale à la valeur `temperature`. - `"NEUMANN"` : Paroi adiabatique isolée ($\partial T/\partial n = 0$).

3.2.6. Module `immersed_boundary` (Pénalisation de Brinkman) Permet d'insérer des obstacles solides rigides immobiles dans l'écoulement : - `enabled` (booléen, défaut : `false`) : Active la pénalisation. - `type` (chaîne) : - `"BOX"` : Obstacle parallélépipédique (marche descendante, cavité en équerre). Paramètres : `box_x0`, `box_y0`, `box_x1`, `box_y1`. - `"CYLINDER"` : Obstacle circulaire plein (cylindre immergé). Paramètres : `cylinder_xc`, `cylinder_yc`, `cylinder_r`. - `permeability_solid` (réel, défaut : `1.0e-40`) : Perméabilité artificielle de Brinkman K_{solid} [m^2].

3.2.7. Module `initial_condition` (Champs Initiaux) Définit la distribution initiale des variables à $t = 0$: - `type` (chaîne) : - `"UNIFORM"` : Vitesse initiale uniforme $(\mathbf{u}_0, \mathbf{v}_0)$. - `"POISEUILLE"` : Profil parabolique établi imposé comme condition initiale. - `"DROPLET"` : Goutte circulaire de phase 1 dans une matrice de phase 2 (`center_x`, `center_y`, `radius`). - `"BUBBLE"` : Bulle circulaire de phase 2 dans un liquide lourd de phase 1. - `"DAM_BREAK"` : Colonne d'eau initiale définie par un rectangle $[x_0, x_1] \times [y_0, y_1]$. - `"VORTEX_SHEAR"` : Disque fluide soumis à un cisaillement de déformation (Cas 3.19). - `"RAYLEIGH_TAYLOR"` : Stratification dense au-dessus d'un fluide léger avec perturbation ondulatoire.

3.2.8. Module `solver` (Solveurs Algébriques et Lagrangien Augmenté)
- `type` (chaîne, défaut : `"AUGMENTED_LAGRANGIAN"`) : Méthode de couplage vitesse-pression.
- `linear_solver` (chaîne, défaut : `"BICGSTAB2"`) :
  - `"BICGSTAB2"` : Bi-Conjugate Gradient Stabilized d'ordre 2 (optimal).
  - `"BICGSTAB"` : BiCGSTAB standard.
  - `"CG"` : Gradient Conjugué (réservé aux matrices symétriques définies positives comme le Laplacien).
  - `"GMRES"` : Generalized Minimal Residual avec redémarrages.
- `precond_type` (chaîne, défaut : `"JACOBI"`) :
  - `"JACOBI"` : Préconditionnement diagonal (très performant et idéal en MPI).
  - `"SSOR"` : Sur-relaxation successive symétrique.
  - `"ILU0"` : Factorisation LU incomplète sans remplissage.
- `tol` (réel, défaut : `1.0e-5`) : Critère d'arrêt sur le résidu relatif $\|\mathbf{r}\|/\|\mathbf{b}\|$.
- `max_iter` (entier, défaut : `300`) : Nombre maximal d'itérations de Krylov autorisées par pas.
- `r_al` (réel, défaut : `500.0`) : Paramètre de pénalité de divergence r [kg/(m$^3 \cdot$ s)].
- `uzawa_iter` (entier, défaut : `1`) : Nombre d'itérations d'Uzawa par pas de temps.

3.2.9. Module `output` (Sauvegardes et Visualisation)
- `directory` (chaîne) : Nom du dossier où sont enregistrés les fichiers de sortie.
- `vtk_interval` (entier) : Intervalle d'écriture des fichiers ParaView `.pvtr` / `.vtr` (en nombre de pas).
- `csv_interval` (entier) : Intervalle d'écriture dans l'historique scalaire `.csv`.
- `console_interval` (entier) : Intervalle de rafraîchissement des informations dans le terminal.

---

4. Tutoriel pas-à-pas : Construire un Cas de Simulation

4.1. Tutoriel 1 : Écoulement Monophasique dans une Cavité Entraînée ($Re = 1000$)

Dans ce cas classique, un fluide visqueux est enfermé dans une cavité carrée de 1 m $\times$ 1 m. Le couvercle supérieur se déplace à vitesse horizontale $u_{lid} = 1.0$ m/s. La viscosité est ajustée à $\mu = 10^{-3}$ Pa$\cdot$s avec une densité $\rho = 1$ kg/m^3 pour obtenir $Re = \frac{\rho U L}{\mu} = 1000$.

Créez le fichier `cases/mon_cas_cavite.json` :

```
{
  "case_name": "cavite_tutoriel",
  "mesh": {
    "nx": 128,
    "ny": 128,
    "lx": 1.0,
    "ly": 1.0
  },
  "physics": {
    "enable_convection": true,
```

```
    "enable_two_phase": false,
    "enable_surface_tension": false,
    "gx": 0.0,
    "gy": 0.0,
    "fluid1": {
      "rho": 1.0,
      "mu": 0.001
    }
  },
  "time": {
    "dt": 0.05,
    "t_final": 40.0,
    "adaptive_dt": false
  },
  "boundary_conditions": {
    "left":   { "type": "DIRICHLET", "u": 0.0, "v": 0.0 },
    "right":  { "type": "DIRICHLET", "u": 0.0, "v": 0.0 },
    "bottom": { "type": "DIRICHLET", "u": 0.0, "v": 0.0 },
    "top":    { "type": "DIRICHLET", "u": 1.0, "v": 0.0 }
  },
  "solver": {
    "type": "AUGMENTED_LAGRANGIAN",
    "linear_solver": "BICGSTAB2",
    "precond_type": "JACOBI",
    "tol": 1.0e-5,
    "max_iter": 300,
    "r_al": 100.0,
    "uzawa_iter": 1
  },
  "output": {
    "directory": "output_tutoriel_cavite",
    "vtk_interval": 100,
    "console_interval": 20
  }
}
```

Lancez le calcul sur 4 cœurs MPI :

```
mpirun -np 4 bin/covof2d cases/mon_cas_cavite.json
```

---

5. Exécution du Code en Séquentiel et Parallèle (OpenMPI)

5.1. Lancement Local sur Poste de Travail

Pour exécuter un cas en séquentiel :

```
bin/covof2d cases/mon_cas.json
```

Pour exécuter en parallèle sur plusieurs cœurs physiques de votre machine :

```
mpirun -np 2 bin/covof2d cases/mon_cas.json
mpirun -np 8 bin/covof2d cases/mon_cas.json
```

5.2. Lancement sur Supercalculateur et Gestionnaire de Tâches SLURM

Sur un cluster de calcul administré par SLURM, créez un script de soumission `job_covof2d.sh` :

```
#!/bin/bash
#SBATCH --job-name=covof2d_calc
```

```
#SBATCH --output=covof2d_%j.log
#SBATCH --error=covof2d_%j.err
#SBATCH --nodes=2
#SBATCH --ntasks-per-node=32
#SBATCH --time=04:00:00
#SBATCH --partition=standard

module purge
module load gcc/11.2.0 openmpi/4.1.2 cmake/3.22.1

export OMP_NUM_THREADS=1

srun bin/covof2d cases/cavity_re1000.json
```

Soumettez la tâche :

```
sbatch job_covof2d.sh
```

5.3. Décomposition de Domaine MPI et Équilibrage de Charge

COVOF 2D divise automatiquement la grille cartésienne globale $N_x \times N_y$ en une grille de sous-domaines $P_x \times P_y = NP$ de façon à minimiser le rapport périmètre/surface (minimisation des volumes de communication inter-processus).

Chaque processus MPI gère son propre sous-domaine local bordé par une rangée de cellules fantômes (*halos*) qui sont synchronisées de façon asynchrone par `MPI_Isend` et `MPI_Irecv` à chaque étape de produit matrice-vecteur et à chaque itération temporelle.

---

6. Suivi d'Exécution et Diagnostic des Calculs en Temps Réel

6.1. Interprétation du Tableau de Bord Console

Lors de l'exécution, COVOF 2D affiche périodiquement un tableau synthétique :

```
Pas      Temps [s]     dt [s]       Iter Krylov Vitesse Max     Div Max          Nu Moyen
-------------------------------------------------------------------------------------------
50       2.50000       5.000e-02    78          9.4120e-01      3.1205e-04       2.7261
100      5.00000       5.000e-02    64          9.4350e-01      1.8410e-04       2.7261
```

- **Pas** : Numéro du pas de temps courant.
- **Temps [s]** : Temps physique accumulé depuis le début de la simulation.
- **dt [s]** : Pas de temps appliqué.
- **Iter Krylov** : Nombre d'itérations nécessaires au solveur BiCGSTAB(2) pour atteindre la tolérance résiduelle.
- **Vitesse Max [m/s]** : Norme infinie $\|\mathbf{u}\|_\infty$ du champ de vitesse.
- **Div Max [s ^{1}]** : Norme infinie de la divergence discrète $\|\nabla \cdot \mathbf{u}\|_\infty$. Elle doit rester inférieure à 10^{-3} s^{-1} pour garantir la stricte incompressibilité physique.
- **Nu Moyen** : Nombre de Nusselt moyen à la paroi chaude (affiché si le module thermique est activé).

6.2. Détection Précoce des Instabilités et Divergences

1. **Explosion de la Vitesse Max** : Si la vitesse dépasse brutalement plusieurs fois la vitesse caractéristique du problème, la condition CFL a été enfreinte ou le terme convectif centré est devenu instable.
2. **Stagnation des itérations Krylov à `max_iter`** : Indique un mauvais conditionnement de la matrice, généralement résolu en augmentant le paramètre de pénalité r_{al} ou en diminuant Δt.
3. **Div Max anormalement élevée (> 0.1)** : Le paramètre de pénalité r_{al} est trop faible, ou un plus grand nombre d'itérations d'Uzawa (`uzawa_iter = 2`) est requis.

---

7. Post-Traitement et Analyse Graphique avec ParaView

7.1. Chargement des Fichiers `.pvtr` et `.vtr`

1. Lancez ParaView (`paraview`).
2. Cliquez sur **File -> Open...** et naviguez vers le dossier de résultats (ex : `output_cavity_re1000`).
3. Sélectionnez le groupe de fichiers maîtres : `cavity_re1000_..pvtr`.
4. Cliquez sur **Apply** dans le panneau *Properties*.

7.2. Extraction des Lignes de Courant et Profils de Vitesse

— **Lignes de courant** :

1. Sélectionnez votre source dans l'arbre d'objets.
2. Appliquez le filtre **Stream Tracer**.
3. Choisissez le vecteur `Velocity`, positionnez la ligne source de graines (Seed Type : High Resolution Line Source) et cliquez sur **Apply**.

— **Profils axiaux 1D** :

1. Appliquez le filtre **Plot Over Line**.
2. Définissez les coordonnées de début $(0.5, 0.0)$ et de fin $(0.5, 1.0)$.
3. Cliquez sur **Apply** pour visualiser la courbe $u(y)$ à mi-cavité.

---

8. Automatisation des Benchmarks et Scripts de Vérification

COVOF 2D est livré avec une suite d'outils Python et Bash permettant de reproduire l'ensemble des cas tests académiques en un clic :

8.1. Le Script d'Orchestration Globale `run_all_benchmarks.sh`

```
# Exécution complète de toute la suite avec OpenMPI sur 2 processus :
bash scripts/run_all_benchmarks.sh 2
```

Ce script compile le code, vide les anciens répertoires de sortie, exécute successivement les 13 cas tests, lance le script de vérification d'erreur et génère l'ensemble des figures.

8.2. Vérification Statistique Automatisée `verify_cases.py`

```
python3 scripts/verify_cases.py
```

Lit les résultats numériques dans les dossiers `output_*`, calcule les normes d'erreur L_1, L_2, L_∞, les confronte aux valeurs de référence du Cahier de Validation et Vérification CFD 2D et produit un tableau récapitulatif `validation_summary.csv` et `validation_summary.md`.

8.3. Production Automatique des Figures de Rapport `generate_validation_figures.py`

```
python3 scripts/generate_validation_figures.py
```

Reconstitue les sous-domaines MPI partagés, extrait les champs scalaires et vectoriels et trace les 14 figures de référence enregistrées dans `docs/figures/`.

---

9. Foire Aux Questions (FAQ) et Astuces Pratiques

Q : Comment choisir au mieux le paramètre de pénalité r_{al} ?
R : En règle générale, choisissez $r_{al} \approx 100 \times \frac{\mu}{\rho}$ pour des régimes laminaires standards, ou $r_{al} \approx \frac{\rho \Delta x^2}{\Delta t}$ pour des calculs transitoires fins. Une valeur trop petite nuit à l'incompressibilité (div $\mathbf{u} > 0$), tandis qu'une valeur démesurée ($r_{al} > 10^7$) dégrade le conditionnement de la matrice.

Q : Pourquoi privilégier BiCGSTAB(2) plutôt que GMRES ?
R : BiCGSTAB(2) ne requiert le stockage que de quelques vecteurs de travail (empreinte mémoire minime et fixe), tout en éliminant les décrochages de convergence propres au BiCGSTAB simple. GMRES nécessite de stocker la base de Krylov complète, ce qui augmente fortement la consommation de RAM pour de grands maillages.

Q : Comment modéliser un profil de vitesse pulsé ou instationnaire à l'entrée ?
R : Consultez le Manuel Développeur pour surcharger la méthode `apply_velocity_boundary_conditions` dans `src/MpiComm.cpp`.

---

10. Guide Détaillé de Dépannage et Traitement des Erreurs

Lors de calculs complexes sur maillages fins ou à haut nombre de Reynolds / Rayleigh, des difficultés numériques peuvent subvenir. Ce tableau répertorie les 12 cas d'incidents les plus fréquents et leur protocole de résolution :

Symptôme / Message d'Erreur	Cause Numérique ou Physique	Diagnostic Immédiat	Action de Remédiation Recommandée
`Krylov solver failed to converge (max_iter reached)`	Conditionnement dégradé de la matrice	Le résidu $\|\mathbf{r}\|/\|\mathbf{b}\|$ stagne au-dessus de `tol`.	1. Réduire le pas de temps `dt` d'un facteur 2.2. Augmenter le paramètre de pénalité `r_al` (ex : passer de 100 à 500).3. Augmenter `max_iter` (passer de 300 à 600).4. Basculer de `BICGSTAB` à `BICGSTAB2` ou `GMRES`.
`Floating Point Exception (NaN or Inf detected)`	Dépassement brutal de la limite de stabilité CFL	Vitesse ou gradient explose vers l'infini.	1. Vérifier la vitesse imposée aux frontières.2. Activer `adaptive_dt = true` avec `cfl_max = 0.3`.3. Basculer le schéma convectif de `CENTERED` vers `UPWIND` ou `TVD_SUPERBEE`.
`Divergence max reste élevée (> 0.01 s^-1)`	Incompressibilité insuffisante	La contrainte $\nabla \cdot \mathbf{u} = 0$ n'est pas satisfaite.	1. Augmenter `uzawa_iter` à 2 ou 3.2. Augmenter `r_al` d'un facteur 10.3. Resserrer la tolérance du solveur `tol` à 10^{-6}.
`Oscillations parasites de l'interface VOF`	Violation de la restriction capillaire de Brackbill	Ondes de haute fréquence déforment la bulle/goutte.	1. Vérifier que $\Delta t \leq \sqrt{\frac{(\rho_1+\rho_2)\Delta x^3}{4\pi\sigma}}$.2. Réduire `dt` d'un facteur 2.3. Vérifier que la tension de surface `sigma` est réaliste.
`Perte de masse excessive de la bulle (> 3%)`	Diffusion numérique ou schéma convectif d'ordre 1	Le volume de la phase VOF diminue au cours du temps.	1. Raffiner le maillage spatial dans la zone interfaciale.2. Utiliser le limiteur `TVD_SUPERBEE` pour préserver la netteté du front.
`MPI Abort / Deadlock à l'échange de halos`	Blocage dans les communications point-à-point	Un processus attend un message non émis.	1. Vérifier la parité de la grille cartésienne MPI.2. S'assurer que le nombre de cellules locales $N_x^{loc} \geq 2$ par processus.

Symptôme / Message d'Erreur	Cause Numérique ou Physique	Diagnostic Immédiat	Action de Remédiation Recommandée
`Vitesse non nulle dans un obstacle solide immergé`	Perméabilité de Brinkman K_{solid} trop élevée	Le fluide pénètre à l'intérieur du corps solide.	1. Vérifier que `permeability_solid` est fixée à 10^{-40} m^2.2. Vérifier la fonction d'appartenance géométrique (`BOX` ou `CYLINDER`).
`Profil de température instable ou T < T_cold`	Dispersion thermique ou condition limite incohérente	Valeurs hors bornes thermodynamiques.	1. S'assurer que les conditions limites thermiques respectent le principe du maximum.2. Utiliser une discrétisation implicite pour l'énergie thermique.
`Segmentation Fault au démarrage`	Dépassement de la mémoire RAM disponible	La grille globale $N_x \times N_y$ excède la capacité mémoire.	1. Augmenter le nombre de processus MPI pour répartir l'empreinte mémoire.2. Réduire la taille de la grille $N_x \times N_y$.
`Nombre de Nusselt oscillant ou divergent`	Maillage insuffisant dans la couche limite thermique	Le gradient pariétal $\partial T/\partial x$ est mal résolu.	1. Raffiner la grille cartésienne en X pour disposer d'au moins 8 mailles dans la couche limite thermique $\delta_T \sim LRa^{-1/4}$.
`Courants parasites importants autour d'une goutte statique`	Déséquilibre entre gradient de pression et force CSF	Des vortex artificiels apparaissent autour de la goutte au repos.	1. Vérifier que la formulation Balanced-Force est active.2. S'assurer que la grille est strictement carrée ($\Delta x = \Delta y$).
`Erreur de lecture JSON (SyntaxError or Missing Key)`	Format JSON corrompu ou virgule orpheline	Le parseur rejette le fichier de configuration.	1. Valider le fichier JSON avec l'utilitaire `python3 -m json.tool cases/mon_cas.json`.

---

11. Annexe Exhaustive : Fiches de Paramétrage des 13 Cas Tests

Chaque cas de la suite de validation Référence Académique CFD dispose d'un fichier JSON complet et prêt à l'emploi dans le répertoire `cases/` :

11.1. Cas 3.1 : `cases/laplacian.json` (Laplacien Cartésien $\nabla^2\psi = 1$)

```
{
  "case_name": "laplacian",
  "mesh": { "nx": 64, "ny": 32, "lx": 2.0, "ly": 1.0 },
  "physics": {
    "enable_convection": false,
    "enable_two_phase": false,
    "enable_surface_tension": false,
    "gx": 0.0, "gy": 0.0,
    "fluid1": { "rho": 1.0, "mu": 1.0 }
  },
```

```
  "time": { "dt": 1.0, "t_final": 1.0, "adaptive_dt": false },
  "boundary_conditions": {
    "left":   { "type": "DIRICHLET", "u": 0.0, "v": 0.0 },
    "right":  { "type": "DIRICHLET", "u": 0.0, "v": 0.0 },
    "bottom": { "type": "DIRICHLET", "u": 0.0, "v": 0.0 },
    "top":    { "type": "DIRICHLET", "u": 0.0, "v": 0.0 }
  },
  "initial_condition": { "type": "LAPLACIAN_TEST" },
  "solver": {
    "type": "AUGMENTED_LAGRANGIAN",
    "linear_solver": "CG",
    "precond_type": "JACOBI",
    "tol": 1.0e-12, "max_iter": 500, "r_al": 0.0, "uzawa_iter": 1
  },
  "output": { "directory": "output_laplacian", "vtk_interval": 1, "console_interval": 1 }
}
```

11.2. Cas 3.7 : `cases/poiseuille.json` (Poiseuille Plan Stationnaire)

```
{
  "case_name": "poiseuille",
  "mesh": { "nx": 64, "ny": 32, "lx": 2.0, "ly": 1.0 },
  "physics": {
    "enable_convection": false,
    "enable_two_phase": false,
    "enable_surface_tension": false,
    "gx": 1.0, "gy": 0.0,
    "fluid1": { "rho": 1.0, "mu": 0.1 }
  },
  "time": { "dt": 0.05, "t_final": 15.0, "adaptive_dt": false },
  "boundary_conditions": {
    "left":   { "type": "PERIODIC" },
    "right":  { "type": "PERIODIC" },
    "bottom": { "type": "DIRICHLET", "u": 0.0, "v": 0.0 },
    "top":    { "type": "DIRICHLET", "u": 0.0, "v": 0.0 }
  },
  "initial_condition": { "type": "UNIFORM", "u": 0.0, "v": 0.0 },
  "solver": {
    "type": "AUGMENTED_LAGRANGIAN",
    "linear_solver": "BICGSTAB2",
    "precond_type": "JACOBI",
    "tol": 1.0e-7, "max_iter": 300, "r_al": 100.0, "uzawa_iter": 1
  },
  "output": { "directory": "output_poiseuille", "vtk_interval": 50, "console_interval": 20 }
}
```

11.3. Cas 3.8 : `cases/poiseuille_unsteady.json` (Poiseuille Transitoire de Szymanski)

```
{
  "case_name": "poiseuille_unsteady",
  "mesh": { "nx": 32, "ny": 64, "lx": 0.02, "ly": 0.02 },
  "physics": {
    "enable_convection": false,
    "enable_two_phase": false,
    "enable_surface_tension": false,
    "gx": 5.55e-5, "gy": 0.0,
    "fluid1": { "rho": 1000.0, "mu": 1.0e-3 }
```

```
  },
  "time": { "dt": 0.01, "t_final": 2.0, "adaptive_dt": false },
  "boundary_conditions": {
    "left":   { "type": "PERIODIC" },
    "right":  { "type": "PERIODIC" },
    "bottom": { "type": "DIRICHLET", "u": 0.0, "v": 0.0 },
    "top":    { "type": "DIRICHLET", "u": 0.0, "v": 0.0 }
  },
  "initial_condition": { "type": "UNIFORM", "u": 0.0, "v": 0.0 },
  "solver": {
    "type": "AUGMENTED_LAGRANGIAN",
    "linear_solver": "BICGSTAB2",
    "precond_type": "JACOBI",
    "tol": 1.0e-6, "max_iter": 300, "r_al": 1000.0, "uzawa_iter": 1
  },
  "output": { "directory": "output_poiseuille_unsteady", "vtk_interval": 20, "console_interval": 10 }
}
```

11.4. Cas 3.9 : `cases/poiseuille_variable_viscosity.json` (Viscosité Exponentielle)

```
{
  "case_name": "poiseuille_variable_viscosity",
  "mesh": { "nx": 32, "ny": 64, "lx": 1.0, "ly": 1.0 },
  "physics": {
    "enable_convection": false,
    "enable_two_phase": false,
    "enable_surface_tension": false,
    "viscosity_model": "EXPONENTIAL",
    "viscosity_exp_mu0": 0.1,
    "viscosity_exp_a": 2.0,
    "viscosity_exp_e": 0.5,
    "gx": 1.0, "gy": 0.0,
    "fluid1": { "rho": 1.0, "mu": 0.1 }
  },
  "time": { "dt": 0.05, "t_final": 10.0, "adaptive_dt": false },
  "boundary_conditions": {
    "left":   { "type": "PERIODIC" },
    "right":  { "type": "PERIODIC" },
    "bottom": { "type": "DIRICHLET", "u": 0.0, "v": 0.0 },
    "top":    { "type": "DIRICHLET", "u": 0.0, "v": 0.0 }
  },
  "initial_condition": { "type": "UNIFORM", "u": 0.0, "v": 0.0 },
  "solver": {
    "type": "AUGMENTED_LAGRANGIAN",
    "linear_solver": "BICGSTAB2",
    "precond_type": "JACOBI",
    "tol": 1.0e-6, "max_iter": 300, "r_al": 100.0, "uzawa_iter": 1
  },
  "output": { "directory": "output_poiseuille_variable_viscosity", "vtk_interval": 40, "console_interval":
}
```

11.5. Cas 3.19 : `cases/vortex_shear_advection.json` (Vortex avec Cisaillement)

```
{
  "case_name": "vortex_shear",
  "mesh": { "nx": 64, "ny": 64, "lx": 1.0, "ly": 1.0 },
  "physics": {
```

```
    "enable_convection": false,
    "enable_two_phase": true,
    "enable_surface_tension": false,
    "convective_scheme": "TVD_SUPERBEE",
    "gx": 0.0, "gy": 0.0,
    "fluid1": { "rho": 1.0, "mu": 1.0 },
    "fluid2": { "rho": 1.0, "mu": 1.0 }
  },
  "time": { "dt": 0.005, "t_final": 4.0, "adaptive_dt": false },
  "boundary_conditions": {
    "left":   { "type": "FREE_SLIP" },
    "right":  { "type": "FREE_SLIP" },
    "bottom": { "type": "FREE_SLIP" },
    "top":    { "type": "FREE_SLIP" }
  },
  "initial_condition": {
    "type": "VORTEX_SHEAR",
    "center_x": 0.5, "center_y": 0.75, "radius": 0.15
  },
  "solver": { "type": "AUGMENTED_LAGRANGIAN" },
  "output": { "directory": "output_vortex_shear", "vtk_interval": 50, "console_interval": 50 }
}
```

11.6. Cas 4.1 : `cases/backward_facing_step.json` (Marche Descendante $Re = 100$)

```
{
  "case_name": "backward_facing_step",
  "mesh": { "nx": 128, "ny": 32, "lx": 40.0, "ly": 2.0612 },
  "physics": {
    "enable_convection": true,
    "enable_two_phase": false,
    "enable_surface_tension": false,
    "gx": 0.0, "gy": 0.0,
    "fluid1": { "rho": 1.0, "mu": 0.01 }
  },
  "immersed_boundary": {
    "enabled": true,
    "type": "BOX",
    "box_x0": 0.0, "box_y0": 0.0, "box_x1": 4.0, "box_y1": 1.0,
    "permeability_solid": 1.0e-40
  },
  "time": { "dt": 0.05, "t_final": 60.0, "adaptive_dt": false },
  "boundary_conditions": {
    "left":   { "type": "DIRICHLET", "u": 1.0, "v": 0.0 },
    "right":  { "type": "NEUMANN" },
    "bottom": { "type": "DIRICHLET", "u": 0.0, "v": 0.0 },
    "top":    { "type": "DIRICHLET", "u": 0.0, "v": 0.0 }
  },
  "initial_condition": { "type": "UNIFORM", "u": 0.0, "v": 0.0 },
  "solver": {
    "type": "AUGMENTED_LAGRANGIAN",
    "linear_solver": "BICGSTAB2",
    "precond_type": "JACOBI",
    "tol": 1.0e-5, "max_iter": 300, "r_al": 100.0, "uzawa_iter": 1
  },
  "output": { "directory": "output_step", "vtk_interval": 100, "console_interval": 50 }
}
```

11.7. Cas 4.4 : `cases/obstacle_cylinder_re20.json` (Cylindre Immergé $Re = 20$)

```
{
  "case_name": "cylinder_re20",
  "mesh": { "nx": 64, "ny": 64, "lx": 1.0, "ly": 1.0 },
  "physics": {
    "enable_convection": true,
    "enable_two_phase": false,
    "enable_surface_tension": false,
    "gx": 0.0, "gy": 0.0,
    "fluid1": { "rho": 1.0, "mu": 0.01 }
  },
  "immersed_boundary": {
    "enabled": true,
    "type": "CYLINDER",
    "cylinder_xc": 0.5, "cylinder_yc": 0.5, "cylinder_r": 0.1,
    "permeability_solid": 1.0e-40
  },
  "time": { "dt": 0.05, "t_final": 15.0, "adaptive_dt": false },
  "boundary_conditions": {
    "left":   { "type": "DIRICHLET", "u": 1.0, "v": 0.0 },
    "right":  { "type": "NEUMANN" },
    "bottom": { "type": "FREE_SLIP" },
    "top":    { "type": "FREE_SLIP" }
  },
  "initial_condition": { "type": "UNIFORM", "u": 1.0, "v": 0.0 },
  "solver": {
    "type": "AUGMENTED_LAGRANGIAN",
    "linear_solver": "BICGSTAB2",
    "precond_type": "JACOBI",
    "tol": 1.0e-5, "max_iter": 300, "r_al": 100.0, "uzawa_iter": 1
  },
  "output": { "directory": "output_cylinder_re20", "vtk_interval": 50, "console_interval": 20 }
}
```

11.8. Cas 4.5 : `cases/cavity_re1000.json` (Cavité Carrée Entraînée $Re = 1000$)

```
{
  "case_name": "cavity_re1000",
  "mesh": { "nx": 128, "ny": 128, "lx": 1.0, "ly": 1.0 },
  "physics": {
    "enable_convection": true,
    "enable_two_phase": false,
    "enable_surface_tension": false,
    "gx": 0.0, "gy": 0.0,
    "fluid1": { "rho": 1.0, "mu": 0.001 }
  },
  "time": { "dt": 0.1, "t_final": 80.0, "adaptive_dt": false },
  "boundary_conditions": {
    "left":   { "type": "DIRICHLET", "u": 0.0, "v": 0.0 },
    "right":  { "type": "DIRICHLET", "u": 0.0, "v": 0.0 },
    "bottom": { "type": "DIRICHLET", "u": 0.0, "v": 0.0 },
    "top":    { "type": "DIRICHLET", "u": 1.0, "v": 0.0 }
  },
  "initial_condition": { "type": "UNIFORM", "u": 0.0, "v": 0.0 },
  "solver": {
    "type": "AUGMENTED_LAGRANGIAN",
```

```
    "linear_solver": "BICGSTAB2",
    "precond_type": "JACOBI",
    "tol": 1.0e-5, "max_iter": 300, "r_al": 100.0, "uzawa_iter": 1
  },
  "output": { "directory": "output_cavity_re1000", "vtk_interval": 200, "console_interval": 50 }
}
```

11.9. Cas 4.7 : `cases/cavity_l_shaped.json` (Cavité en Forme de L $Re = 1000$)

```
{
  "case_name": "cavity_l_shaped",
  "mesh": { "nx": 64, "ny": 64, "lx": 1.0, "ly": 1.0 },
  "physics": {
    "enable_convection": true,
    "enable_two_phase": false,
    "enable_surface_tension": false,
    "gx": 0.0, "gy": 0.0,
    "fluid1": { "rho": 1.0, "mu": 0.001 }
  },
  "immersed_boundary": {
    "enabled": true,
    "type": "BOX",
    "box_x0": 0.5, "box_y0": 0.5, "box_x1": 1.0, "box_y1": 1.0,
    "permeability_solid": 1.0e-40
  },
  "time": { "dt": 0.05, "t_final": 15.0, "adaptive_dt": false },
  "boundary_conditions": {
    "left":   { "type": "DIRICHLET", "u": 0.0, "v": 0.0 },
    "right":  { "type": "DIRICHLET", "u": 0.0, "v": 0.0 },
    "bottom": { "type": "DIRICHLET", "u": 0.0, "v": 0.0 },
    "top":    { "type": "DIRICHLET", "u": 1.0, "v": 0.0 }
  },
  "initial_condition": { "type": "UNIFORM", "u": 0.0, "v": 0.0 },
  "solver": {
    "type": "AUGMENTED_LAGRANGIAN",
    "linear_solver": "BICGSTAB2",
    "precond_type": "JACOBI",
    "tol": 1.0e-5, "max_iter": 300, "r_al": 100.0, "uzawa_iter": 1
  },
  "output": { "directory": "output_cavity_l", "vtk_interval": 50, "console_interval": 20 }
}
```

11.10. Cas 6.1 : `cases/natural_convection_ra1e4.json` (Convection Naturelle $Ra = 10^4$)

```
{
  "case_name": "natural_convection_ra1e4",
  "mesh": { "nx": 64, "ny": 64, "lx": 1.0, "ly": 1.0 },
  "physics": {
    "enable_convection": true,
    "enable_two_phase": false,
    "enable_surface_tension": false,
    "gx": 0.0, "gy": -9.81,
    "fluid1": { "rho": 1.0, "mu": 0.01 }
  },
  "thermal": {
    "enable_energy": true,
    "prandtl": 0.71,
```

```
    "rayleigh": 1.0e4,
    "beta": 0.00341,
    "t_ref": 0.0,
    "t_hot": 1.0,
    "t_cold": 0.0,
    "k_thermal": 0.01408,
    "cp_thermal": 1.0
  },
  "time": { "dt": 0.05, "t_final": 20.0, "adaptive_dt": false },
  "boundary_conditions": {
    "left":   { "type": "DIRICHLET", "u": 0.0, "v": 0.0, "thermal_type": "DIRICHLET", "temperature": 1.0 }
    "right":  { "type": "DIRICHLET", "u": 0.0, "v": 0.0, "thermal_type": "DIRICHLET", "temperature": 0.0 }
    "bottom": { "type": "DIRICHLET", "u": 0.0, "v": 0.0, "thermal_type": "NEUMANN" },
    "top":    { "type": "DIRICHLET", "u": 0.0, "v": 0.0, "thermal_type": "NEUMANN" }
  },
  "initial_condition": { "type": "UNIFORM", "u": 0.0, "v": 0.0 },
  "solver": {
    "type": "AUGMENTED_LAGRANGIAN",
    "linear_solver": "BICGSTAB2",
    "precond_type": "JACOBI",
    "tol": 1.0e-5, "max_iter": 300, "r_al": 100.0, "uzawa_iter": 1
  },
  "output": { "directory": "output_natural_convection_ra1e4", "vtk_interval": 50, "console_interval": 20 }
}
```

11.11. Cas 7.1 : `cases/dam_break.json` (Rupture de Barrage à Surface Libre)

```
{
  "case_name": "dam_break",
  "mesh": { "nx": 100, "ny": 50, "lx": 2.0, "ly": 1.0 },
  "physics": {
    "enable_convection": true,
    "enable_two_phase": true,
    "enable_surface_tension": false,
    "gx": 0.0, "gy": -9.81,
    "fluid1": { "rho": 1000.0, "mu": 1.0e-3 },
    "fluid2": { "rho": 1.0,    "mu": 1.8e-5 }
  },
  "time": { "dt": 0.001, "t_final": 1.5, "adaptive_dt": false },
  "boundary_conditions": {
    "left":   { "type": "DIRICHLET", "u": 0.0, "v": 0.0 },
    "right":  { "type": "DIRICHLET", "u": 0.0, "v": 0.0 },
    "bottom": { "type": "DIRICHLET", "u": 0.0, "v": 0.0 },
    "top":    { "type": "FREE_SLIP" }
  },
  "initial_condition": {
    "type": "DAM_BREAK",
    "box_x0": 0.0, "box_y0": 0.0, "box_x1": 0.5, "box_y1": 0.5
  },
  "solver": {
    "type": "AUGMENTED_LAGRANGIAN",
    "linear_solver": "BICGSTAB2",
    "precond_type": "JACOBI",
    "tol": 1.0e-5, "max_iter": 200, "r_al": 500.0, "uzawa_iter": 1
  },
  "output": { "directory": "output_dam_break", "vtk_interval": 50, "console_interval": 20 }
}
```

11.12. Cas 7.4 : `cases/static_droplet.json` (Goutte Statique et Loi de Laplace)

```
{
  "case_name": "static_droplet",
  "mesh": { "nx": 64, "ny": 64, "lx": 1.0, "ly": 1.0 },
  "physics": {
    "enable_convection": false,
    "enable_two_phase": true,
    "enable_surface_tension": true,
    "sigma": 1.0,
    "gx": 0.0, "gy": 0.0,
    "fluid1": { "rho": 1.0, "mu": 1.0 },
    "fluid2": { "rho": 1.0, "mu": 1.0 }
  },
  "time": { "dt": 0.005, "t_final": 1.0, "adaptive_dt": false },
  "boundary_conditions": {
    "left":   { "type": "PERIODIC" },
    "right":  { "type": "PERIODIC" },
    "bottom": { "type": "PERIODIC" },
    "top":    { "type": "PERIODIC" }
  },
  "initial_condition": {
    "type": "DROPLET",
    "center_x": 0.5, "center_y": 0.5, "radius": 0.25
  },
  "solver": {
    "type": "AUGMENTED_LAGRANGIAN",
    "linear_solver": "BICGSTAB2",
    "precond_type": "JACOBI",
    "tol": 1.0e-5, "max_iter": 200, "r_al": 500.0, "uzawa_iter": 1
  },
  "output": { "directory": "output_droplet", "vtk_interval": 20, "console_interval": 10 }
}
```

11.13. Cas 7.5 : `cases/rising_bubble.json` (Remontée de Bulle Hysing)

```
{
  "case_name": "rising_bubble",
  "mesh": { "nx": 60, "ny": 120, "lx": 1.0, "ly": 2.0 },
  "physics": {
    "enable_convection": true,
    "enable_two_phase": true,
    "enable_surface_tension": true,
    "sigma": 1.96,
    "gx": 0.0, "gy": -0.98,
    "fluid1": { "rho": 1000.0, "mu": 10.0 },
    "fluid2": { "rho": 100.0,  "mu": 1.0 }
  },
  "time": { "dt": 0.002, "t_final": 3.0, "adaptive_dt": false },
  "boundary_conditions": {
    "left":   { "type": "FREE_SLIP" },
    "right":  { "type": "FREE_SLIP" },
    "bottom": { "type": "DIRICHLET", "u": 0.0, "v": 0.0 },
    "top":    { "type": "DIRICHLET", "u": 0.0, "v": 0.0 }
  },
  "initial_condition": {
    "type": "BUBBLE",
```

```
    "center_x": 0.5, "center_y": 0.5, "radius": 0.25
  },
  "solver": {
    "type": "AUGMENTED_LAGRANGIAN",
    "linear_solver": "BICGSTAB2",
    "precond_type": "JACOBI",
    "tol": 1.0e-5, "max_iter": 200, "r_al": 1000.0, "uzawa_iter": 1
  },
  "output": { "directory": "output_bubble", "vtk_interval": 50, "console_interval": 20 }
}
```